

PROJECT DOCUMENTATION

Date	02/07/2026
Team ID	
Project Name	Book Store
Maximum Marks	4 Marks

1. Introduction

- **Project Title:** Bookstore Management System (MERN Stack)
- **Team Members:** [As Specified]

2. Project Overview

Purpose: The Bookstore Management System is a robust web application developed using the MERN Stack (MongoDB, Express.js, React.js, and Node.js). It allows users to seamlessly browse through available book catalogs, view comprehensive description metrics, manage an active digital shopping cart, and view backend inventory statuses via an interactive interface.

Features:

- Dynamic Home Landing Dashboard View
- Comprehensive Book Inventory Listings
- Categorized Navigation System (Fiction, Finance, Technology, Self-Help, Romance, Mystery)
- Active Session Client-Side Shopping Cart Handler
- Responsive Interface Design Across Alternative Device Formats
- Client-Side React Routing Interceptor Engine
- Persistent NoSQL Storage Layer Integration (MongoDB)
- REST API Schema Framework Supported by Express.js Routing

Goals:

- Provide a low-latency web architecture for online book navigation.
- Render structured catalog records elegantly within dynamic front-end grid view frameworks.
- Support modular state mutations for shopping cart operations.
- Demonstrate robust end-to-end multi-tier system decoupling under the MERN convention.

3. Architecture

Frontend Tier: Fabricated cleanly atop React.js and styled seamlessly utilizing React Bootstrap components. Operates as a single-page decoupled system executing optimized view mutations without page refreshes.

- **Core Modules:** Home Dashboard, Global Navigation Header, Book Gallery, Cart Manager, Application Footer.
- **Routing Parameters:** React Router elements drive state routing on the client side.

Backend Tier: Engineered using Node.js runtime layers bound to Express.js micro-routing handlers. Governs standard REST query handling, database channel serialization, payload integrity checks, and handles output stream conversions into structured JSON records.

Database Layer: MongoDB stores persistent document formats under clean structural schemas:

```
Books Collection Schema Mapping:
{
  "_id": "ObjectId",
  "title": "String",
  "author": "String",
  "category": "String",
  "price": "Number",
  "image": "String",
  "description": "String"
}
```

4. Setup Instructions

Prerequisites: Maintain local operational instances of Node.js, MongoDB Community Server, npm engine packages, and an IDE editor workspace Environment.

Installation Commands:

```
# Clone target repository
git clone <repository-link>

# Initialize frontend dependencies
cd client
npm install

# Initialize backend server packages
cd server
npm install
```

Configuration Parameters: Establish an explicit `.env` file footprint inside your `/server` runtime context:

```
PORT=5000
MONGO_URI=your_mongodb_connection_string
```

5. Folder Structure

```
client/
├─ public/
└─ src/
    ├─ Components/
    ├─ Pages/
    ├─ App.js
    └─ index.js

server/
├─ models/
├─ routes/
├─ controllers/
└─ server.js
```

6. Running the Application

Server Launch sequence: Run `cd server && npm start` to spin up the endpoint router on port 5000. Concurrently execute `cd client && npm start` to configure local client compilation targeting `http://localhost:3000`.

7. API Documentation

Get All Available Catalog Items: `GET /api/books`

```
Response Payload Format:
[
  {
    "title": "The Alchemist",
    "author": "Paulo Coelho",
    "price": 499
  }
]
```

Operational Routes: Supports GET /api/books/:id (Target Item Extraction), POST /api/books (Catalog Insertion), PUT /api/books/:id (Record Alteration), and DELETE /api/books/:id (Document Erasure).

8. Authentication Status

"Authentication and authorization are not implemented in the current version of the project. The application is publicly accessible for browsing books."

9. User Interface & Testing

Testing methodologies verify clean route-link navigation mapping, precise API payload status checks, successful NoSQL persistence commits, active cart increment rules, and layout stability benchmarks across alternative device viewports using Developer Consoles and Postman systems.

11. Screenshots or Demo

The integrated user interface modules captured from the running application instance match the functional screen components below:

Hero Welcome Screen Banner View

Figure 1.1: Hero Welcome Screen Banner View ("Discover Your Next Favorite Book")

Featured Browse Grid Categories View

Figure 1.2: Featured Browse Grid Categories View & Value Props Section

Best Seller Inventory Showcase Grid View

Figure 1.3: Best Seller Inventory Showcase Grid View with Active Database Records

Top Recommendation Carousel Grid Feed View

Figure 1.4: Top Recommendation Carousel Grid Feed View

12. Known Issues & Future Enhancements

- **Current Limitations:** Shopping cart data arrays persist temporarily inside client-side local runtime variables and clear upon application reloads; an admin dashboard controller framework is not integrated.
- **Future Enhancements Blueprint:** Integration of tokenized JWT user authentication modules, third-party transactional gateway integration (Stripe API), automated product review ranking grids, and multi-faceted search filtration indexes.